Table des matières

Synthèse — Comptoir Aurore, une base lue par son journal 1
Étape 0 — La boutique, et ce que vous allez bâtir 1
Étape 1 — Générer, et monter le projet 2
Étape 2 — Le premier regard 3
Étape 3 — La RAW, telle que livrée 5
Étape 4 — La base, rebâtie de son journal 6
Étape 5 — Les événements web, un flux sans courtier 8
Étape 6 — Le contrat 10
Étape 7 — Une commande, et la preuve 11
Étape 8 — Allonger la période 13
Récapitulatif 13
Annexe — L'arborescence et les commandes 15

Synthèse — Comptoir Aurore, une base lue par son journal

Synthèse avancée — Formation Data Engineering · l'arc des Cours 01 à 06, 09 et 11, d'un bloc

Étape 0 — La boutique, et ce que vous allez bâtir

Comptoir Aurore inc. est une boutique en ligne de Montréal. Elle vend des articles de maison, d'électronique et de plein air au Canada depuis 2020, et en France depuis le 4 mars 2024. Sa base de vente est une vraie base PostgreSQL, dix-huit tables normalisées, et personne ne vous donnera accès à ses tables. Ce que vous recevez, c'est le journal de ses changements, tel qu'un connecteur Debezium l'a publié : chaque insertion, chaque modification, chaque suppression, depuis la mise en service de la plateforme le 1er janvier 2023. À côté, les événements de l'outil d'analyse du site web, et les taux de change de la Banque du Canada.

Là où La Brûlerie tenait dans une caisse, Aurore réunit ce qui rend un pipeline réel difficile. Le journal publie sans schéma, et une date y est un nombre. Le connecteur a redémarré un soir et republié sept heures de changements. Une colonne s'est ajoutée à la table des commandes en cours de route. Le flux web livre au moins une fois, en retard parfois, a changé de format en juin 2025, et porte un robot que personne n'a filtré. Vous allez refaire l'arc de la formation d'un bloc, sur ce décor :

les fichiers livrés → `RAW`, avec son journal des chargements → la base rebâtie de son journal, schéma `boutique` → les événements web dédoublonnés, schéma `argent` → le contrat, le tout lancé par une seule commande, et prouvé contre la copie de la base que la boutique livre à côté.

© 2026 iAlexMG. Tous droits réservés.

Dossier des sources	Ce qu'il porte	Ici
`appli/journal/`	le journal de capture, un fichier de messages par jour UTC	la source de la base
`appli/postgresql/`	la copie de la base au soir de la date de fin, et son script de chargement	le schéma de la base, et le témoin
`web/`	les événements du site, un fichier par jour UTC de réception	le flux
`change/`	les taux quotidiens de l'euro en dollars canadiens, réels	chargés tels quels
`avis/`	les avis des clients	laissés à AI Engineering

Ce document ne réexplique rien : chaque étape nomme la section de cours qui fonde son geste, et ne garde que ce qui est propre à Aurore. Il se fait après le mini-cas de La Brûlerie, une fois le Cours 12 derrière vous. Il se fait en deux séances, coupées après l'étape 4 : la première lit le journal et rebâtit la base, la seconde traite le flux web, écrit le contrat et prouve le tout.

Le corrigé est dans `Solution/01_de/`, le dépôt tel qu'il doit être à la fin. Chaque bloc de ce document qui appartient au dépôt en est tiré, sans ses commentaires d'en-tête.

Étape 1 — Générer, et monter le projet

La configuration hors du code est au Cours 05 (`Theorie_05_DE.md`, section 2), et l'empaquetage en une seule porte d'entrée à la section 7.

Depuis `Data_Domain`, dans l'Anaconda Prompt, env `formations` actif :

```
mkdir Projet_Aurore\01_de
python _univers/aurore/generer_aurore.py --projet Projet_Aurore
cd Projet_Aurore\01_de
git init
```

Le dossier du projet se crée d'abord : le générateur refuse d'écrire dans un dossier qui n'existe pas. Attendu, en 45 secondes environ : 2 256 fichiers, et la ligne « Conforme à la référence des cours ». Si vous avez fait la synthèse Aurore d'Analytics Engineering, les sources sont déjà là : le générateur les refait à l'identique, à la même empreinte. `Projet_Aurore/` est ignoré par Git, comme `Projet_Boreal/`.

© 2026 iAlexMG. Tous droits réservés.

Dossier	Ce qu'il porte	Qui l'écrit
`Projet_Aurore/sources/`	la sortie du générateur, jamais retouchée	le générateur
`Projet_Aurore/plateforme/entrepot/aurore.duckdb`	les schémas `raw`, `boutique`, `argent` et `pipeline`	le pipeline
`Projet_Aurore/01_de/`	le dépôt : le paquet `aurore`, ses scripts SQL, son test	vous

Le dépôt porte un `config.toml`, et le code n'en connaît aucun chemin :

```
[chemins]
sources = "sources"
entrepot = "plateforme/entrepot/aurore.duckdb"
```

Et un paquet, `aurore`, bâti comme `brulerie` : une porte d'entrée, `__main__.py`, un module par étape, et `config.py`, qui lit ce fichier et retrouve le projet, le parent du dépôt. Les modules `config.py`, `entrepot.py` et `fenetre.py` sont ceux du mini-cas, à deux détails près dans la fenêtre : elle n'a plus à lister ses jours, faute d'option `--par-jour`, et le lot d'un fichier d'Aurore est un jour seul, sans l'instant d'un export.

Étape 2 — Le premier regard

DuckDB pour regarder sans charger est au Cours 01 (`Theorie_01_DE.md`, section 6). La capture de changements par le journal de la base est au Cours 04 (`Theorie_04_DE.md`, section 3.3). Lire un champ d'un objet JSON, `->>`, est au Cours 08 de la formation SQL avancé, section 4.

Le journal est fait de messages Kafka, un par ligne, dans des fichiers compressés. Les ouvrir un à un ne dirait rien : `read_json_objects` lit chaque ligne comme un objet JSON, sans deviner de colonnes. Dans la ligne de commande DuckDB, lancée depuis `Projet_Aurore` :

```
select json->>'$.value.op' as op, count(*) as messages
from read_json_objects('sources/appli/journal/*/*.jsonl.gz', format = 'newline_delimited')
group by op
order by op;
```

op	Ce que c'est	Messages
`r`	une lecture de l'instantané initial, le 2023-01-01	7 045
`c`	une création	108 740
`u`	une modification	48 830

© 2026 iAlexMG. Tous droits réservés.

op	Ce que c’est	Messages
d	une suppression	35

Voici un message du 9 octobre 2024, tel qu’il est livré, sur une seule ligne :

```
{"topic":"aurore.boutique.expedition","partition":0,"offset":11168,"key":{"exp_id":705566},
 "value":{"before":{"exp_id":705566},
          "after":{"exp_id":705566,"cmd_id":505526,"ent_id":1,"transporteur":"POSTES_CANADA",
                   "statut":"LIVREE","expedie_le":"2024-10-07T18:36:43Z",
                   "livre_le":"2024-10-08T18:16:59Z"},
          "source":{..., "ts_ms":1728411419000, "table":"expedition", "lsn":392745296, ...},
          "op":"u","ts_ms":1728435960000}}
```

Quatre choses s’y lisent, et le fichier `sources/appli/journal/LISEZMOI.md` les confirme.

Ce qu’on voit	Ce que ça veut dire
`before` ne porte que `exp_id`	la table est en `REPLICA IDENTITY DEFAULT` : la ligne d’avant se réduit à la clé. Cinq tables seulement, `client`, `adresse`, `commande`, `produit` et `retour`, portent toute la ligne d’avant
des montants comme `"115.20"`, entre guillemets	`decimal.handling.mode` vaut `string` : un décimal voyage en texte, pour rester exact
aucun schéma dans le message	`schemas.enable` vaut `false` : une colonne `date` y est un nombre de jours depuis le 1970-01-01, et rien dans le message ne dit que c’en est une
`source.ts_ms` le 8 octobre à 18 h 16 UTC, `ts_ms` le 9 à 1 h 06	le changement a été validé la veille, et traité sept heures plus tard : c’est un message republié

Le connecteur a redémarré le soir du 8 octobre 2024 et republié les changements lus de 18 h à 1 h UTC, sous de nouvelles positions (A5). Kafka livre au moins une fois. Ce qui reconnaît un doublon, c’est la ligne visée et sa position dans le journal des transactions de PostgreSQL, `source.lsn` :

```sql
select count(*) as changements_publies_deux_fois
from (
    select json->>'topic', json->>'key', json->>'$.value.source.lsn'
    from read_json_objects('sources/appli/journal/*/*.jsonl.gz', format = 'newline_delimited')
    group by all
    having count(*) > 1
);
```

Attendu : 127. Puis cherchez le premier fichier où une commande porte le mode de livraison :

© 2026 iAlexMG. Tous droits réservés.

```sql
select min(filename) as premier_fichier
from read_json_objects('sources/appli/journal/*/*.jsonl.gz', format = 'newline_delimited',
                       filename = true)
where json_exists(json, '$.value.after.mode_livraison');
```

C'est `changements_2025-01-20.jsonl.gz` : la colonne a été ajoutée ce jour-là (A6), et aucun message d'avant ne la porte.

Le flux web, lui, se lit avec `read_json`, parce que ses champs sont stables d'un fichier à l'autre, à une exception près, qui justifie `union_by_name` :

```sql
select export_version, count(*) as evenements, count(distinct event_id) as identifiants,
       min(received_at) as premier, max(received_at) as dernier
from read_json('sources/web/*/*.jsonl.gz', union_by_name = true)
group by export_version
order by export_version;
```

Version	Événements	Identifiants	Reçus du	Au
1	307 316	306 042	2023-01-01	2025-06-01
2	100 758	100 363	2025-06-02	2025-12-31

Deux faits sortent de ce tableau. Il y a plus d'événements que d'identifiants : 1 669 doublons de livraison. Et le format a changé le 2 juin 2025 (A7) : la version 2 renomme `device_category` en `device` et ajoute `consent_mode`. Sans `union_by_name`, DuckDB devine les colonnes sur les premiers fichiers, tous en version 1 : la colonne `device` n'existe pas, et l'appareil des 100 758 événements de la version 2 disparaît sans une erreur.

Étape 3 — La `RAW`, telle que livrée

Le chargement incrémental est au Cours 03 (`Theorie_03_DE.md`, section 2), l'idempotence et le journal des chargements à la section 3, la couche `RAW` à la section 4.

Le chargeur est celui du mini-cas : une empreinte SHA-256 par fichier, le journal `pipeline.chargements`, une transaction qui écrit la table et le journal ensemble. Ce qui change, c'est la façon de lire un fichier de JSON lines : un message par ligne, entier, dans une colonne `JSON`. La `RAW` ne l'interprète pas.

```python
LIVRAISONS = [
    ("journal", "sources/appli/journal/*/changements_*.jsonl.gz", "json"),
    ("web_evenements", "sources/web/*/evenements_*.jsonl.gz", "json"),
    ("change_fxeurcad", "sources/change/FXEURCAD.csv", "csv"),
]

# Le lot se lit dans le nom : « changements_2024-10-09.jsonl.gz » est du 2024-10-09.
LOT = re.compile(r"_(\d{4}-\d{2}-\d{2})\.")
```


© 2026 iAlexMG. Tous droits réservés.

```
LECTEURS = {
    "json": "SELECT json AS message, filename FROM read_json_objects($fichiers, "
            "format = 'newline_delimited', filename = true)",
    "csv": "SELECT * FROM read_csv($fichiers, header = true, all_varchar = true, filename = true)",
}
```

Garder le message entier est un choix. Il coûte de la place, 172 Mo pour tout l'entrepôt, et il rend la `RAW` rejouable : quand une étape suivante se trompe, on corrige la requête et on relit la `RAW`, sans jamais redemander un fichier à la source. Le journal y garde ses 127 messages republiés et le web ses 1 669 doublons, parce que c'est ce qui a été livré.

Le taux de change, lui, arrive en un seul fichier, `FXEURCAD.csv`, sans date dans son nom : son lot est `complet`, et c'est son empreinte qui dit s'il a changé. Il entre tout en texte, 767 jours, et la synthèse s'arrête là : la conversion des devises appartient à Analytics Engineering.

Étape 4 — La base, rebâtie de son journal

Lire une base par son journal est au Cours 04 (`Theorie_04_DE.md`, section 3.3). L'ordre des messages d'une même clé est au Cours 09 (`Theorie_09_DE.md`, section 8), et « au moins une fois, puis idempotent » à la section 13.

Rebâtir la base, c'est rejouer le journal : pour chaque ligne de chaque table, le dernier changement connu donne son état, à moins que ce soit une suppression. Deux questions restent : ce qui est « dernier », et le type de chaque colonne.

Le dernier changement d'une ligne est celui qui a la plus haute position dans le journal des transactions, `source.lsn`, et non celui qui est arrivé le dernier. Une ligne, c'est un sujet et une clé :

```
DDL = re.compile(r"^CREATE TABLE .*?^\);", re.S | re.M)

# Le dernier changement de chaque ligne : une ligne, c'est un sujet et une clé.
DERNIERS = """
CREATE OR REPLACE TEMP TABLE derniers AS
SELECT message->>'$.value.source.table' AS table_source,
       message->>'$.value.op' AS op,
       message->'$.value.after' AS apres
FROM raw.journal
QUALIFY row_number() OVER (
    PARTITION BY message->>'$.topic', message->>'$.key'
    ORDER BY (message->>'$.value.source.lsn')::BIGINT DESC,
             (message->>'$.offset')::BIGINT DESC) = 1"""
```


© 2026 iAlexMG. Tous droits réservés.

Un message republié porte le même `lsn` que le premier, donc le même changement : il ne change rien à l'état. Le rejeu est idempotent par construction, sans dédoublonnage explicite. Ce qui ne l'est pas, c'est tout ce qui compte des changements : combien de fois une commande a changé de statut en octobre 2024 se trompe de 127 si l'on compte les messages plutôt que les couples clé et `lsn`.

Sur ce journal, lire les messages dans l'ordre d'arrivée aurait donné le même état : mesuré, 0 écart sur les 115 667 lignes, parce qu'aucune ligne republiée n'a changé de nouveau entre sa publication et sa republication. Ce n'est pas une raison de s'en passer. Dans un sujet, 7 messages arrivent après un message plus récent qu'eux : il aurait suffi qu'une de ces lignes change pendant les sept heures pour que l'ordre d'arrivée rende un état périmé, sans rien signaler.

Reste le type. Le connecteur publie sans schéma, et le seul endroit qui dit qu'une colonne est une date, c'est le DDL de la base. La boutique le livre avec la copie, dans `appli/postgresql/charger_appli.sql`. La synthèse d'Analytics Engineering a montré que ce DDL passe tel quel dans DuckDB : le pipeline n'en garde que les `CREATE TABLE`, recrée le schéma `boutique` avec, puis lit les types dans le catalogue de DuckDB et en tire chaque colonne du message :

```
  def extraire(colonne: str, type_sql: str) -> str:
      """L'expression qui tire une colonne de la ligne « after », au type que le DDL lui donne."""
      valeur = f"apres->>'{colonne}'"
      if type_sql == "DATE":
          # time.precision.mode = adaptive : une date voyage en jours depuis le 1970-01-01.
          return f"DATE '1970-01-01' + CAST({valeur} AS INTEGER)"
      return f"CAST({valeur} AS {type_sql})"
```

Un montant en texte devient un `DECIMAL` exact par un simple `CAST`, un instant ISO 8601 à l'heure UTC un `TIMESTAMPTZ`. Seule la date demande un calcul. Le test `tests/test_etat.py` vérifie quatre cas sur une base en mémoire, sans fichier : le jour 20000 donne le 2024-10-04, `"145.75"` reste 145,75, une colonne absente du message vaut `NULL`, et le lot d'un fichier se lit dans son nom.

Ce dernier cas est celui d'A6. Les commandes d'avant le 20 janvier 2025 n'ont jamais porté `mode_livraison`, et leur dernier message ne l'a pas : la colonne sort vide. Mesuré sur la base rebâtie, 6 872 commandes sans valeur, 4 328 à domicile et 399 en point relais. Le vide ne veut pas dire « livré à domicile » : il veut dire « d'avant la colonne », et c'est à Analytics Engineering d'en décider.

La boucle qui rebâtit les dix-huit tables tient en quelques lignes, dans une transaction qui commence par effacer le schéma :

```
        for creation in DDL.findall(script):
            con.execute(creation)
        con.execute(DERNIERS)
        tables = catalogue(con)
        lignes = 0
        for table, colonnes, types in tables:
            select = ", ".join(extraire(c, t) for c, t in zip(colonnes, types))
            con.execute(f"INSERT INTO boutique.{table} SELECT {select} FROM derniers "
                        "WHERE table_source = ? AND op <> 'd'", [table])
```

La base se refait en entier à chaque passage : 164 650 messages se rejouent en une seconde environ, plus vite qu'une fusion jour par jour ne s'écrirait (`Theorie_06_DE.md`, section 5). Deux autres faits du journal passent sans code : les 35 suppressions portent toutes sur `adresse`, dont celle de la cliente qui a

© 2026 iAlexMG. Tous droits réservés.

demandé l'effacement de ses données le 2025-11-04 (A9), et un message supprimé n'est suivi d'aucun message vide, `tombstones.on.delete` étant à `false`. Une ligne supprimée disparaît simplement de l'état.

Piège — un `->>` qui suit un `AND` se lit mal. DuckDB lit `x = 'commande' and message->'$.value.after' is not null` comme `(x = 'commande' and message)->'$.value.after'`, et plante sur une conversion. Placé en tête d'une expression, `message->>'...' = 'x'` se lit juste. Au moindre doute, parenthésez : `(message->>'$.value.source.table') = 'commande'`. Le Labo 11 a rencontré le même piège, de l'autre côté d'un signe égal.

À retenir — la ligne d'avant ne sert pas à rebâtir l'état. Treize tables sur dix-huit ne publient que la clé dans `before`, et l'état se rebâtit quand même, du seul `after`. Elle servirait à tenir l'historique de ce qui a changé ; pour ces treize tables, cet historique se tire de votre propre état précédent, jamais du message.

Étape 5 — Les événements web, un flux sans courtier

Ce que « livrer » veut dire est au Cours 09 (`Theorie_09_DE.md`, section 4), le temps de l'événement, les fenêtres et les retards à la section 11, et « exactement une fois, en effet » à la section 13. Côté langage, les CTE sont au Cours 05 de la formation SQL avancé, sections 2 et 3, `QUALIFY row_number()` à son Cours 04, sections 3 et 7, et `JOIN … USING` à son Cours 01, section 7. L'instant, son epoch et le fuseau de session qui décide du jour sont à son Cours 02, section 6.

Le flux web arrive en fichiers quotidiens, et non par un courtier. La logique d'un consommateur ne change pas pour autant : reconnaître un doublon, choisir son heure, décider quoi faire du retard. `sql/argent_web.sql` refait `argent.web_evenements` à partir de la `RAW`, et commence par les doublons :

```
WITH uniques AS (
    -- Un doublon de livraison reprend le même event_id, reçu de 2 à 30 secondes après le premier.
    SELECT message
    FROM raw.web_evenements
    QUALIFY row_number() OVER (PARTITION BY message->>'event_id'
                               ORDER BY message->>'received_at') = 1
),
```

Un doublon se reconnaît à son identifiant, jamais à son contenu : ses deux copies sont identiques à l'heure de réception près, et une comparaison des contenus ne les rapprocherait pas. Mesuré : 408 074 événements livrés, 406 405 gardés, 1 669 écartés, chacun reçu deux fois au plus.

Les deux versions du format se ramènent à un seul nom par `coalesce(message->>'device', message->>'device_category')`. Le reste du script ne fait que typer : l'heure de l'événement et celle de la réception en `TIMESTAMPTZ`, la valeur d'un achat en `DECIMAL`, le numéro de la commande en entier.

Puis le retard. L'heure qui compte pour l'analyse est celle de l'événement ; le fichier du jour, lui, suit la réception. Mesuré sur les 406 405 événements :

Retard entre l'événement et sa réception	Événements
5 secondes au plus	404 097

© 2026 iAlexMG. Tous droits réservés.

Retard entre l’événement et sa réception	Événements
de 1 minute à 1 heure	5
plus d’une heure, jusqu’à 10 heures	2 303

Les 2 303 retardataires sont tous des mobiles, 989 visites faites hors réseau et envoyées à la reconnexion. Une fenêtre quotidienne, au jour UTC de l’événement, qui se fermerait à minuit manquerait 561 événements. Le filigrane du Cours 09 chiffre le prix de l’attente :

Filigrane après minuit UTC	Événements encore en retard
aucun	561
1 heure	456
6 heures	95
10 heures	0

Ici, la question se règle d’elle-même, et c’est un choix plus qu’une chance : la table se refait en entier depuis la `RAW` à chaque passage, donc un événement en retard retrouve son jour dès le passage suivant. C’est la troisième réponse du Cours 09, rouvrir la fenêtre et corriger le résultat, qui ne coûte rien quand le résultat se recalcule de toute façon.

Reste le robot (A8). Il se voit au premier regard sur les visites : 264 visites de 40 à 80 événements, tous des pages de produit, en moins de quatre minutes chacune, du 9 au 20 septembre 2024, sans un achat ni une connexion. La plus longue visite humaine de la période compte 11 événements. Le seuil tombe entre les deux :

```sql
SELECT e.*,
       epoch(e.recu_le - e.survenu_le) AS retard_s,
       -- Le robot d'A8 : des visites de 40 à 80 pages de produit en moins de quatre minutes. Une
       -- visite humaine n'atteint jamais 20 événements. L'outil web ne le filtre pas : on le
       -- signale, et c'est à BI et à DS de décider s'il sort de leurs chiffres.
       v.evenements >= 20 AND v.types = 1 AS robot
FROM e
JOIN visites AS v USING (visite_id);
```

Mesuré : 264 visites et 15 694 événements, exactement ceux que le générateur a plantés. Signaler plutôt que retirer n’est pas de la prudence : un entonnoir de conversion veut écarter le robot, un suivi de charge du site veut le garder, et l’ingénieur de données ne sait pas lequel des deux lira la table.

Piège — le fuseau de la session. Un `TIMESTAMPTZ` converti en date prend le fuseau de la session DuckDB, celui du poste : `America/Toronto` sur un poste réglé à l’heure de Montréal. Un événement reçu à 1 h UTC le 2 juin tombe alors le 1er juin, et le jour de réception ne correspond plus au fichier. Le contrat de l’étape suivante écrit donc `CAST(recu_le AT TIME ZONE 'UTC' AS DATE)`, en toutes lettres.

© 2026 iAlexMG. Tous droits réservés.

Un dernier recoupement sert la suite du parcours : les 6 738 événements `purchase` citent 6 738 commandes distinctes, toutes présentes dans la base rebâtie, sur 11 599. Le site ne mesure que les visites qui ont accepté les témoins, et Business Intelligence partira de cet écart.

Étape 6 — Le contrat

Le contrat est au Cours 11 (`Theorie_11_DE.md`, section 2), les dimensions d'une règle à la section 3, le choix de bloquer, d'avertir ou de suivre à la section 4, et la version d'un contrat à la section 5. Les règles s'écrivent en SQL : `NOT EXISTS` est au Cours 07 de la formation SQL avancé, section 4, et `UNION ALL`, qui les empile, à la section 6 du même cours.

`sql/controles_aurore.sql` rend une ligne par règle, avec son niveau. Chaque promesse d'une source y devient une règle, et chaque règle un compte :

Règle	Source	Niveau	Attendu
positions sans trou dans chaque sujet	journal	bloque	0
un message republié porte le même changement	journal	bloque	0
colonnes connues du schéma de la base	journal	bloque	0
versions connues du format web	web	bloque	0
champs de la version du format	web	bloque	0
identifiants d'événement uniques	web, après dédoublonnage	bloque	0
achats d'une commande de la base	web contre base	bloque	0
doublons de livraison écartés	web	suit	1 669
événements reçus un autre jour UTC	web	suit	561
événements du robot, signalés	web	suit	15 694

Une règle qui suit ne signale pas une faute de la source. Un robot, un doublon ou un retard sont la vie normale d'un site ; on les compte pour voir le jour où ils changent d'échelle. Les règles qui bloquent, elles, protègent ce que les étapes précédentes ont supposé.

© 2026 iAlexMG. Tous droits réservés.

La première dit plus qu'elle n'en a l'air. Des positions sans trou depuis 0 prouvent que la `RAW` part de l'instantané initial : une base rebâtie d'un journal pris en cours de route serait fausse sans rien signaler. Mesuré sur un projet dont la `RAW` ne portait que 2026, la règle bloque sur 12 sujets, tous ceux qui ont reçu un message en 2026.

Deux autres gardent les événements A6 et A7 :

```
SELECT 'colonnes connues du schéma de la base', 'bloque', count(*)
FROM colonnes_du_journal AS j
WHERE NOT EXISTS (SELECT 1 FROM duckdb_columns() AS d
                  WHERE d.schema_name = 'boutique' AND d.table_name = j.table_source
                    AND d.column_name = j.colonne)
UNION ALL
SELECT 'versions connues du format web', 'bloque', count(*)
FROM raw.web_evenements
WHERE message->>'export_version' NOT IN ('1', '2')
```

La première attrape une colonne qui arriverait dans le journal avant d'arriver dans le DDL. `extraire` ne lit que les colonnes du DDL : sans cette règle, la colonne neuve serait perdue en silence. Mesuré sur une copie du projet dont le DDL avait perdu `mode_livraison`, comme si la boutique avait livré le schéma d'avant le 20 janvier 2025 :

```
ERROR     règle rompue : colonnes connues du schéma de la base, 1 écart(s)
ERROR   1 règle(s) bloquante(s) du contrat en écart.
```

La seconde, dans sa première version, ne connaissait que la version 1 du format. Mesuré sur un projet chargé jusqu'au 1er juin 2025 avec cette règle, puis au 2 juin :

```
ERROR     règle rompue : versions connues du format web, 430 écart(s)
ERROR   1 règle(s) bloquante(s) du contrat en écart.
```

Les 430 événements du 2 juin sont tous en version 2. C'est le cas du Cours 11 : le contrat ne se contourne pas en baissant la sévérité de la règle, il se renégocie. La version 2 renomme un champ, un changement qui casse ; le contrat passe donc de 1.0.0 à 2.0.0, qui déclare les deux versions, et la règle « champs de la version du format » dit ce que chacune doit porter. Relancé du 2 juin jusqu'au bout avec le contrat à jour, le projet passe, témoin compris.

Bloquer, ici, veut dire rendre le code 1 et nommer la règle. Ce que l'exécution a déjà écrit reste en place, et c'est le rejeu de la même tranche, une fois le contrat renégocié, qui le refait.

Étape 7 — Une commande, et la preuve

La porte d'entrée unique est au Cours 05 (`Theorie_05_DE.md`, section 7). L'intervalle de données est au Cours 06 (`Theorie_06_DE.md`, section 3).

© 2026 iAlexMG. Tous droits réservés.

Une exécution enchaîne les quatre étapes, et le témoin sur demande :

```
  def une_execution(con, fenetre, avec_temoin: bool) -> None:
      debut = time.monotonic()
      LOG.info("exécution %s", fenetre)
      raw.executer(con, fenetre)
      etat.executer(con, chemin("sources"))
      web.executer(con)
      controles.executer(con)
      if avec_temoin:
          temoin.executer(con, chemin("sources"))
      LOG.info("fin, en %.1f s", time.monotonic() - debut)
```

Le témoin compare la base rebâtie à la copie de la base que la boutique livre au soir de la date de fin, table par table, valeur par valeur, dans les deux sens (`EXCEPT ALL`). C'est la preuve qu'une capture de changements reconstruit bien ce qu'elle prétend reconstruire. Depuis `Projet_Aurore/01_de` :

```
  pytest
  python -m aurore --depuis 2023-01-01 --jusqua 2025-12-31 --temoin
```

La période est de 2023 à 2025, et pourtant le témoin rompt :

```
  ERROR     témoin adresse : 1 ligne(s) de la copie absentes, 0 en trop
  ERROR     témoin client : 2 ligne(s) de la copie absentes, 1 en trop
  ERROR     témoin commande : 5 ligne(s) de la copie absentes, 0 en trop
  ERROR     témoin commande_ligne : 6 ligne(s) de la copie absentes, 0 en trop
  ERROR     témoin commande_taxe : 8 ligne(s) de la copie absentes, 0 en trop
  ERROR     témoin consentement : 1 ligne(s) de la copie absentes, 0 en trop
  ERROR     témoin paiement : 5 ligne(s) de la copie absentes, 0 en trop
  ERROR   7 table(s) sur 18 diffèrent de la copie de la base.
```

La copie est prise au soir du 31 décembre, heure de Montréal, soit 5 h UTC le 1er janvier. Les fichiers du journal suivent le jour UTC du connecteur : le dernier est `changements_2026-01-01`, 33 messages, dont les cinq commandes passées le soir du 31. La fenêtre s'arrêtait un jour trop tôt. Chargez ce jour-là :

```
  python -m aurore --depuis 2026-01-01 --jusqua 2026-01-01 --temoin
```

`pytest` rend `4 passed`. Attendu du pipeline : les 18 tables égalent la copie, 115 667 lignes, les comptes de la section 2 de `_univers/aurore/CHIFFRES_ATTENDUS.md`. Le contrat passe, avec les trois comptes suivis de l'étape 6. La première exécution prend une vingtaine de secondes, et une relance sur toute la période 8 secondes, sans rien charger dans la `RAW`.

La preuve de l'univers dit la même chose par un autre chemin. Depuis `Projet_Aurore/sources` :

```
  python ../../_univers/aurore/preuves/verifier_journal.py .
```


© 2026 iAlexMG. Tous droits réservés.

Attendu : 164 650 messages, dont 127 republiés, et 0 écart sur ses cinq contrôles, positions, instantané, ligne d'avant, doublons et état. Son en-tête dit aussi ce qu'elle ne prouve pas : que le format soit fidèle à celui d'un vrai Debezium, ce qui vient de la documentation de Debezium, pas d'ici.

Étape 8 — Allonger la période

La fenêtre, le journal et la reconstruction sont au Cours 06 (`Theorie_06_DE.md`, section 5).

Trois mois de 2026 s'ajoutent. Depuis `Data_Domain` :

```
python _univers/aurore/generer_aurore.py --projet Projet_Aurore --jusqua 2026-03
```

Un jour déjà produit ne change jamais quand la période s'allonge : seuls changent les fichiers d'état, la copie de la base, le fichier du taux de change, et le dernier fichier du journal. Puis, depuis `Projet_Aurore/01_de`, la tranche neuve, qui va jusqu'au 1er avril UTC :

```
python -m aurore --depuis 2026-01-01 --jusqua 2026-04-01 --temoin
```

Mesuré : 91 fichiers du journal chargés, 14 780 messages, dont le fichier du 1er janvier, qui avait 33 messages et en a maintenant 52. Le journal des chargements l'a reconnu à son empreinte, ses anciennes lignes sont parties, les nouvelles sont entrées. Le taux de change passe de 767 à 829 jours de la même façon. La base rebâtie compte 125 771 lignes et égale la nouvelle copie, et le tout prend 10 secondes.

Le rattrapage n'est une reprise que s'il redonne ce qu'un chargement d'un bloc aurait donné. Mesuré sur trois projets, la période allongée par tranche, la même chargée d'un bloc, et une troisième passée par le blocage d'A7 : 22 tables, 1 191 657 lignes, heure de chargement exclue, 0 écart.

Récapitulatif

Démarche	**Où elle est enseignée**	**Où ici**
la configuration hors du code, la porte d'entrée unique	Cours 05, sections 2 et 7	Étapes 1 et 7
regarder avant de charger	Cours 01, section 6	Étape 2
la `RAW`, son lignage, son journal et ses empreintes	Cours 03, sections 2 à 4	Étape 3
le message gardé entier, sans interprétation	cette synthèse	Étape 3

© 2026 iAlexMG. Tous droits réservés.

Démarche	Où elle est enseignée	Où ici
la capture de changements par le journal de la base	Cours 04, section 3.3	Étapes 2 et 4
l’ordre d’une clé, la position qui décide	Cours 09, section 8	Étape 4
le type qui vient du DDL, faute de schéma dans le message	cette synthèse	Étape 4
au moins une fois, puis idempotent	Cours 09, section 13	Étapes 4 et 5
le temps de l’événement, la fenêtre, le filigrane	Cours 09, section 11	Étape 5
signaler plutôt que retirer	Cours 11, section 4	Étapes 5 et 6
un contrat qui bloque, qui suit, et qui change de version	Cours 11, sections 2 à 5	Étape 6
le témoin contre l’état de la source, et le jour UTC qui déborde	cette synthèse	Étape 7
le rattrapage égal au chargement d’un bloc	Cours 06, section 5	Étape 8

Ce que la synthèse ne fait pas, et pourquoi :

Ce qui manque	Où le trouver
Kafka et un consommateur	Cours 09 : le journal et le web arrivent déjà en fichiers de messages, et la logique du consommateur, doublons, ordre et retards, est la même
PostgreSQL et un vrai Debezium	Cours 04, section 3.3 : la copie de la base sert de témoin, et le format suit la documentation de Debezium
l’ordonnanceur	Cours 06 : une commande par jour suffit, et la fenêtre est la sienne
une fusion incrémentale de l’état	nulle part dans cette formation : la base se rebâtit en une seconde, et la rebâtir est plus simple à prouver ; le premier modèle incrémental du parcours est au Cours 07 d’Analytics Engineering
le lac bronze et les tables ouvertes	Cours 02 et 10 : tout tient dans l’entrepôt, et rien n’y est lu par un autre moteur

© 2026 iAlexMG. Tous droits réservés.

Ce qui manque	Où le trouver
les avis des clients	AI Engineering : leur texte libre et leurs instructions déguisées sont son terrain

La base de la boutique n'a jamais été lue, et elle est pourtant là, table par table, égale à sa copie. Le schéma `boutique` porte exactement ce que la synthèse d'Analytics Engineering charge de la copie : une équipe qui recevrait cette base de Data Engineering plutôt que d'un export partirait d'ici.

Annexe — L'arborescence et les commandes

```
Projet_Aurore/
├─ sources/                          la sortie du générateur
├─ plateforme/entrepot/aurore.duckdb raw, boutique, argent, pipeline
└─ 01_de/                            le dépôt de la synthèse
   ├─ config.toml                    les chemins
   ├─ pytest.ini
   ├─ .gitignore
   ├─ aurore/                        __main__, config, fenetre, entrepot, raw, etat, web, controles, temoin
   ├─ sql/                           argent_web.sql, controles_aurore.sql
   └─ tests/test_etat.py
```

Pour tout rejouer, depuis `Projet_Aurore/01_de`, env `formations` actif :

```
pytest
python -m aurore --depuis 2023-01-01 --jusqua 2026-01-01 --temoin
```

Le corrigé complet, commentaires compris, est dans `Solution/01_de/`. Son `README.md` dit comment le rejouer sans rien retaper.

© 2026 iAlexMG. Tous droits réservés.